



Faculty of Engineering & Technology
Electrical & Computer Engineering Department
ADVANCED DIGITAL DESIGN ENCS3310

Report

Prepared by:

Number : 1222275

Name : Anwar Atawna.

Instructor : Abdallatif Abuissa.

Section : 4.

Date : 27/12/2024.

Brief introduction

A comparison system will be made which will be 6 bits unsigned and signed where the inputs will be 6 bits, and the result of the comparison of signed and unsigned will be displayed. This design is considered important in digital systems for decision making and sorting for signal processing.

Brief theoretical overview

A comparison of 6 bits will be made and these comparisons will be as follows:

Unsigned Comparison:

Inputs will be treated as binary numbers that are always positive, where the most significant bit from the left will be compared to the least significant bit from the right.

Where the outputs will be:

GT(Greater Than) : $A > B$.

LW(Less Than) : $A < B$.

EQ(Equal) : $A = B$.

signed Comparison:

Here the inputs will be positive or negative numbers, depending on the MSB. If the MSB is 0, it is a positive number, but if it is 1, it is negative.

Where the outputs will be:

GT(Greater Than) : $A > B$.

LW(Less Than) : $A < B$.

EQ(Equal) : $A = B$.

The basic digital gates (or, and, not, xnor, nor.....) and multiplexers are used to choose whether it is signed or unsigned according to the selection.

Design Philosophy

In order to design the desired design, we must:

The modules :

- Unsigned Comparator Module(unsignedm) :

Here we used the gate to display the result of GT, LW, EQ, as we dealt here by comparing each bit with its counterpart, where the bit on the left is more important than the one on the right, as the importance increases from right to left.

- Signed Comparator Module(signedm):

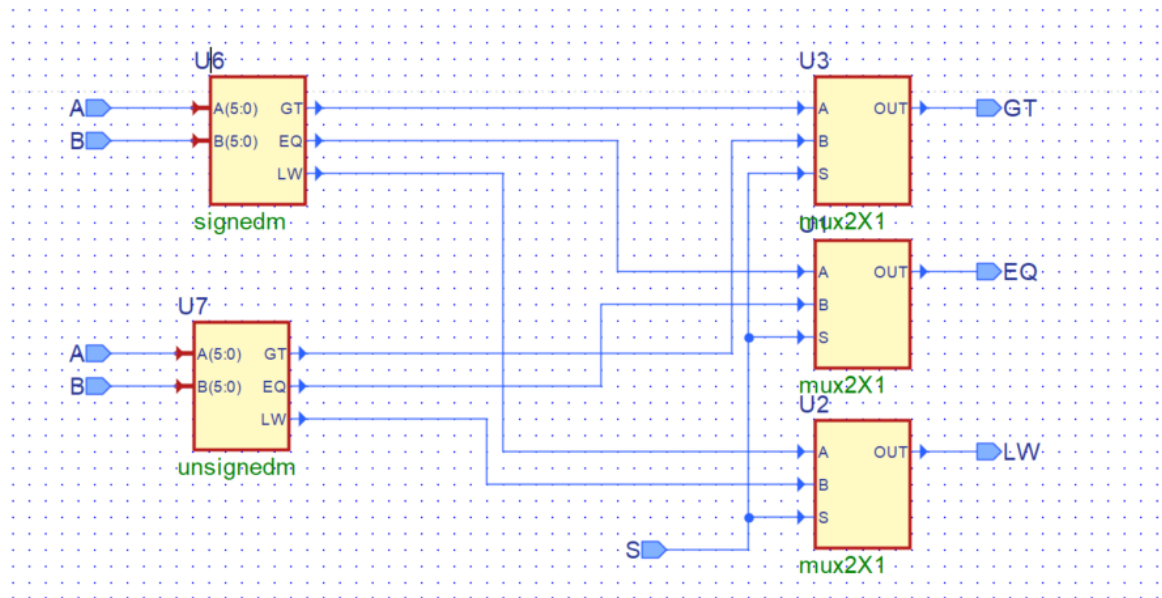
In this module we were concerned with MSB and there were cases:

- Case 1 : (MSB)A = 0 , (MSB)B = 0 => Here the output will be like the unsigned number.
- Case 2 : (MSB)A = 1 , (MSB)B = 0 => Here the output is Less Than (LW = A<B).
- Case 3 : (MSB)A = 0 , (MSB)B = 1 => Here the output is Less Than (LW = A<B).
- Case 4: (MSB)A = 1, (MSB)B = 1 => Here the result will be the opposite of the unsigned number.

Note : Here the unsigned module is called.

- Multiplexer (mux2X1) : Here the multiplexer will be used to determine if ba is signed or unsigned using select if 0 is unsigned else if 1 is signed.
- Structural Design (structural): Here both the unsigned and signed modules are called and the mux is called three times. The signed will give us the values of GT, EQ and LW for the unsigned numbers and for the unsigned values. In each first mux we will put the value of unsigned for GT and signed for GT. According to the select, the final value will be one of them, and so on for EQ, LW.

- Behavioral Design (behavioral): Here we used the behavioral for the if statements. Here we used this module for the test bench comparison between the structural ones.



- Clocked Synchronization: Output synchronization according to the clock signal, which includes operation in serial systems.
- Delay Analysis: The delays on the gates will be translated, which will allow us to calculate the exact time. The time was calculated to be 97 seconds, which corresponds to a frequency of 10.31 MHz.

Simulation Results :

Here the inputs A, B, and S are placed systematically, where the structural and behavioral results will be compared and checked if the results are identical. Here we notice that all the structural values are the same as the behavioral values for both (signed, unsigned).

Here we have some different cases:

- Case 1 :
 - # KERNEL: 41/10000 unsigned :
 - # KERNEL: A = 6 A = 000110 B = 46 B = 101110 S = 0
 - # KERNEL: bGT = 0 bEQ = 0 bLW = 1
 - # KERNEL: sGT = 0 sEQ = 0 sLW = 1
 - # KERNEL: *****PASS*****

- Inputs: A = 6 (000110), B = 46 (101110), S = 0 (Unsigned mode)
- Outputs:
 - bGT = 0: A is not greater than B.
 - bEQ = 0: A is not equal to B.
 - bLW = 1: A is less than B.
- Signed Results: Not applicable since unsigned mode is selected.
- Result: The outputs match the expected behavior for unsigned comparison. PASS

- Case 2 :

```
◦ # KERNEL: 50255/0000 signed :  
◦ # KERNEL: A = 16 A = 010000 B = -3 B = 111101 S = 1  
◦ # KERNEL: bGT = 1 bEQ = 0 bLW = 0  
◦ # KERNEL: sGT = 1 sEQ = 0 sLW = 0
```

- Inputs: A = 16 (010000), B = -3 (111101), S = 1 (Signed mode)
- Outputs:
 - bGT = 1: A (positive) is greater than B (negative).
 - sGT = 1: A (positive) is greater than B (negative).
 - bEQ = 0, bLW = 0, sEQ = 0, sLW = 0: The expected result for signed comparison is observed.
- Result: Signed mode correctly identifies A > B. PASS

○ Case 3 :

```
746609000 signed :
A = -8 A = 111000 B = 17 B = 010001 S = 1
bGT = 0 bEQ = 0 bLW = 1
sGT = 0 sEQ = 0 sLW = 1
```

□ Inputs: A = -8 (111000),B = 17 (010001),S = 1 (Signed mode)

□ Outputs:

- bGT = 0: As expected since A appears greater in unsigned representation.
- sGT = 0: Correctly identifies A is not greater than B in signed mode.
- sLW = 1: Correct as A (negative) is less than B (positive).

□ Result: The signed logic accurately handles the negative A compared to positive B.

PASS

○ Case 4 :

```
° # KERNEL: 748092050 signed :
° # KERNEL: A = -8 A = 111000 B = -32 B = 100000 S = 1
° # KERNEL: bGT = 1 bEQ = 0 bLW = 0
° # KERNEL: sGT = 1 sEQ = 0 sLW = 1
```

• Inputs: A = -8 (111000),B = -32 (100000),S = 1 (Signed mode)

• Outputs:

- bGT = 1: Incorrect for signed interpretation.
- sGT = 1: Correct as A (-8) is greater than B (-32) in signed mode.
- bEQ = 0, bLW = 0, sEQ = 0, sLW = 0: Expected outputs for the signed case.

• Result: Despite the negative numbers, the signed logic gives correct results. PASS

Also, if and else statements were placed. If the conditions are identical, print pass. Otherwise, print fail and stop.

Output Logs:

Change the delay time value.

```
◦ # KERNEL: A = 1 A = 000001 B = 0 B = 000000 S = 1
◦ # KERNEL: bGT = 1 bEQ = 0 bLW = 0
◦ # KERNEL: sGT = 0 sEQ = 0 sLW = 1
◦ # KERNEL: FAIL
```

```
initial begin
    {S, A, B} = 0; // Initialize inputs
    repeat (8191) begin
        #40ns {S, A, B} = {S, A, B} + 1; // Increment inputs
        if (GT != sGT || EQ != sEQ || LW != sLW)
            begin
                $display("FAIL");
                $stop; // Stop simulation if outputs mismatch
            end
        else
```

Conclusion and Future Works

This project taught us how to design and simulate a signed and unsigned 6-bit comparator. This ensures a modular design approach and concurrent operation under a variety of conditions. By combining structural and behavioral comparators, we achieve a precise and flexible design of the operation. It will also guide us in the future to expand support to 8-bit or 12-bit widths, for example, and to optimize the critical path to achieve higher operating frequencies.